

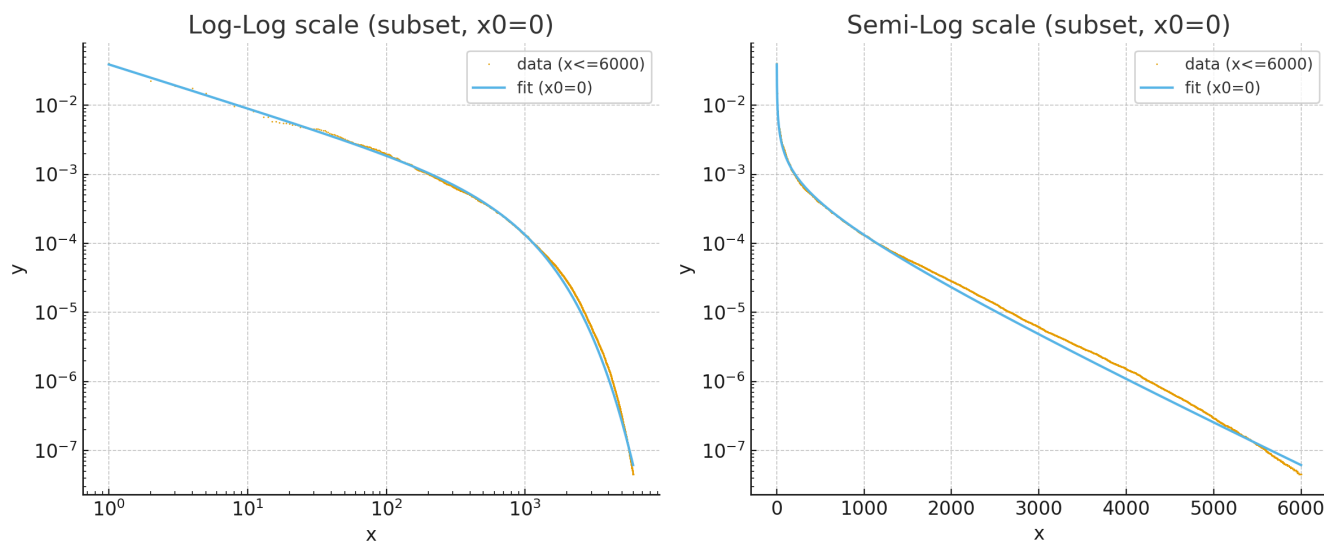
在自然语言中，语言所包含的词的频率和频率排名往往符合 [Zipf 定律](#)：设按从高到低排名，词排第 r 位，则其频率大致正比于 $f(r) = r^{-1}$ ，这种将排名和频率关联的表达式称为「频序-频率关系」。更一般地，可以允许使用不同的幂函数来拟合这个曲线，其一般形式（幂律分布）为

$$f(r) = Cr^{-a}, \quad a > 0, r \in \mathbb{Z}^+$$

然而对于中文来说，情况则比较复杂。一方面，中文词语的确基本符合 Zipf 定律；但另一方面，每个词又由一个或多个汉字构成，因此我们不仅关心词的频率分布，也关心字的频率分布。在现代汉语中，大多数汉字不经常作为词（即一字词）出现，而主要是以多字词的形式出现，这使得字的分布和词的分布有所不同。

幂律-指数分布规律

使用朱宇浩统计的[知乎语料字频](#)，将频序和频率用双对数坐标（下图左）和纵轴对数坐标（下图右）绘制：



如果字也符合 Zipf 定律，那么双对数坐标上应该是一条直线，然而在 $r > 100$ （即前 100 之后的字）即已经出现了比较明显的偏差。而在纵轴对数坐标上可以看到，当 $r > 1000$ 时，频序和频率的对数大致成线性关系，说明字的频率随着频序的增加而指数衰减。因此，幂律分布主导前 100 字，指数分布主导 1000 以后的字，而 100 ~ 1000 则属于过渡阶段。若采用

$$f(r) = Cr^{-a}e^{-r/b}$$

的形式拟合，则可以得到 $a = 0.634$, $b = 765$ 。也就是说，在前 1000 字的范围内大致按照幂律分布快速下降，然后在 1000 字以后则转换为每 700 多字频率下降为之前的 $1/e$ 。实际上，如果保证 $0 < a < 1$ ，则它也可以看成是一种 Gamma 分布。

对这个分布的解释

关于为什么汉字的频率分布和 Zipf 定律有所偏离，[文献](#)给出了一些半定量的解释。

汉字的频率分布不是长尾（重尾）分布

上述的幂律-指数分布满足

$$\lim_{r \rightarrow \infty} e^{\lambda r} f(r) = 0, \quad \forall \lambda \leq 1/b$$

也就是说，它的尾部比指数分布还要少一些。这解释了为什么一些原生选重数特别高的单字方案（例如二码顶）在实际使用中体验仍然不错的现象。

对方案优化的指导意义

重码率的优化

在设计汉字输入方案时，给定一份字频，则可以优化在这个字频上的加权选重率。但考虑到输入方案在不同使用场景下的适用性，一般不能只考虑单一字频下的选重率，而是也要考虑在给定字集内的所有选重的数量（即选重数）。传统意义上，一般优化选重率 p 和选重数 M 的某种加权组合 $p + cM$ ，等价地说，此时频率为 r 的重码字在优化中对目标函数的贡献权重为

$$w(r) = f(r) + c$$

但是，这一方法会有如下问题：

- 若 c 很小，则选重数的权重比较低，对选重数的控制作用不明显。
- 若选重数的权重增大到和选重率大致在同一数量级，则中低频字的频率被拉平，导致中频字和低频字区分不开。例如，如果希望 $\sum_r c \approx 0.1 \sum_r f(r) = 0.1$ ，取总字数为 6763，则 $c = 0.1/6763 \approx 1.5 \times 10^{-5}$ 。观察字频表，把字频表中典型的频率对应频率和这个 c 相加，得到如下结果：

频序 r	频率 $f(r)$	权重 $w(r) = f(r) + c$
1500	5.8×10^{-5}	7.3×10^{-5}
3000	6.2×10^{-6}	2.1×10^{-5}
4500	6.9×10^{-7}	1.5×10^{-5}
6000	4.5×10^{-8}	1.5×10^{-5}

观察到，频序 1500、3000、4500、6000 的字本来频率相差几个数量级，但是加上 c 之后，后面几个的差异基本被抹平了。也就是说，一个频序 3000 的字重码和 6000 的字重码在优化算法看来差不多，而这显然是不符合实际的。也就是说，通过简单地加 c 所获得的兼顾选重数的效果并不好。

那么怎样才能用更合理的方法来兼顾选重率和选重数呢？我们可以从上面发现的幂律-指数分布规律 $f(r) \propto r^{-a}e^{-r/b}$ 入手。

- 首先，从拟合的曲线图中注意到幂律段在前 300 之后的字已经很不明显，且前 300 的字一般不会重码，所以可以直接扔掉幂律项让 $f(r) \propto e^{-r/b}$ 。
 - 对于退火等局域搜索算法来说，这还有一个额外的好处：由于大大降低了高频字的权重，局域搜索的势能面变得更加平滑，比较有利于解在势能面上迁移，提高搜索效率。
- 其次，我们可以通过调节 b 的大小来控制选重率和选重数之间的平衡。 b 的意义是：频序每增加 b 个字，重要性就下降 $e \approx 2.7$ 倍。拟合数据本身得到 $b \approx 750$ ，我们可以先验地把衰减速度开平方，即令 $b = 1500$ 来兼顾中低频的选重。

击键当量的优化

对于编码中包含的按键组合的好坏，传统意义上一般优化以字频加权的键均当量。但有的时候会出现「过拟合」问题：即前 300 字的当量特好（例如 < 1.29 ），而 300 ~ 500 字的当量特差（例如 > 1.36 ）。这是由于前 300 字处于幂律-指数分布的幂律段，权重相当高，算法会不惜一切代价降低这些字的当量；而事实上，如果当量在不同的字之间更加均衡（避免一些特差的组合，而不是尽力给高频字安排特好的组合）实际打字体验好。观察纵轴对数坐标（上图右），发现去掉幂律项、保留指数项可以在优先照顾高频字的前提下，明显降低高频字的「过分的」高权重。因此可以令权重 $w(r) \propto e^{-r/b}$ 其中 $b \approx 750$ 。

总结

1. 现代汉语中汉字字频的频率-频序关系为幂律-指数分布 $f(r) \sim r^{-a}e^{-r/b}$ ，其衰减速度与指数分布相当，而并不是语言学中常见到重尾的 Zipf 分布；换言之，低频字的使用率是相当低的。
2. 在方案优化中，无论是重码还是当量，直接基于字频加权都有一系列的缺点。鉴于采用幂律-指数分布拟合汉字的频率-频序关系效果非常好，我们可以只提取一个字频文件中的频序然后根据需要来重构各个字的权重。我们推荐使用不带幂律项的权重 $w(r) \sim e^{-r/b}$ 来优化。
3. 对于重码和击键当量其权重参数（即衰减速率）可以不相同。我们推荐对于重码使用 $b = 1500$ ，而对于当量使用 $b = 750$ 。